Algorithm 2: ANNS

Input: in-memory navigation graph G_m, PQ short codes, disk-based graph G_d, query q, candidate set C with fixed size, result set R, pruning ratio σ
Output: top-k results of q

1 $S \leftarrow$ entry points of q by a vertex search strategy on G_m;
2 $C \leftarrow S$; ▷ sort by PQ distance to q
3 $R \leftarrow S$; ▷ compute exact distance to q
4 $u \leftarrow$ top-1 unvisited vertex in C; ▷ target vertex
5 $B \leftarrow$ load the block including u from G_d; ▷ DR
6 update C according to u's neighbor IDs; ▷ DC
7 add u to R based on u's full-precision vector; ▷ DC
8 $B' \leftarrow$ top-$((\varepsilon - 1) \cdot \sigma)$ vertices in $B \setminus \{u\}$; ▷ block pruning
9 **while** C *has unvisited vertex* **do**
10 $v \leftarrow$ top-1 unvisited vertex in C; ▷ PQ-based routing
11 conduct line 5 for v, and lines 6–7 for the vertices in B' in parallel;
12 ▷ I/O and computation pipeline
13 conduct lines 6–8 for v and the block including it;
14 **return** top-k vertices in R ▷ sort by exact distance to q

`Starling` performs RS for a query q based on block search. It uses a candidate set C, a result set R, and a kicked set P to store candidate vertices, results, and vertices kicked out from C, respectively. `Starling` changes the length limit of C dynamically to handle different result lengths. The steps of RS are as follows. (1) It obtains the entry points from the in-memory navigation graph and initializes C and R with them. (2) It iteratively explores C and updates C and R (like ANNS). It also adds the unvisited vertices kicked out from C to P. (3) When all the vertices in C are visited, it calculates the ratio of R's length to C's length. Given a ratio threshold φ, if

$$\frac{|R|}{|C|} \geq \varphi \quad , \tag{7}$$

it doubles the size of C and restarts the search. This is because a high ratio means that most candidates are results. In this case, exploring more vertices may find new results. (4) In the next search, `Starling` adds the closer vertices (w.r.t q) in P to C and repeats (2)–(3) for unvisited candidates in C. (5) The search stops when Eq. 7 is not met. After changing the length of C, it resumes the search with the previous results in R, candidates in C, and some closer vertices (w.r.t q) in P. This avoids extra computation and disk access. Our evaluation shows that $\varphi = 0.5$ is optimal.

6 EXPERIMENTS

We evaluate the following aspects of `Starling`: (1) search performance (**§6.2**), (2) I/O-efficiency (**§6.3**), (3) index cost (**§6.4**), (4) ablation study (**§6.5**), (5) parameter sensitivity (**§6.6**), (6) scalability (**§6.7**), (7) query distribution (**§6.8**), (8) segment setup (**§6.9**), (9) large-scale search results (**§6.10**), and (10) billion-scale data (**§6.11**). Our source code and datasets are available at: https://github.com/zilliztech/starling. Kindly refer to Appendix for additional evaluations.

6.1 Experimental Setting

Datasets. We use four public real-world datasets [3] that vary in data type, dimensions, distance, and query type. Tab. 1 shows their details. Unless specified, we limit the raw vectors per segment

Table 1. Statistics of experimental datasets on a segment.

Dataset	Data type	Dimensions	Distance function	# Base vector per segment	# Query	Query type
BIGANN	uint8	128	L2	33M	10^4	ANNS/RS
DEEP	float	96	L2	11M	10^4	ANNS/RS
SSNPP	uint8	256	L2	16M	10^5	RS
Text2image	float	200	IP	5M	10^5	ANNS

to under 4GB and adjust the data scale accordingly for each dataset. We randomly select vectors from standard datasets for a segment. We perform a brute-force search on the selected vectors to get the ground truth. The query sets are the same for a given dataset.

Query workloads. The query workloads' details are provided in Tab. 1. By default, all query sets are derived from real-world scenarios and are not-in-database, meaning they do not have any intersection with the base data. The queries are executed in a random order, and a batch of queries is served using a pool of threads. Each thread is assigned to handle one query at a time.

Compared methods. We compare `Starling` with two state-of-the-art disk-based HVSS methods that are able to process up to 4GB vector data within the 2GB memory constraint. We do not include in-memory methods (e.g., HNSW [49] and IVFPQ [36]) in the evaluation, because they either run out of memory or have very low accuracy, as reported in previous works [16, 35, 60].

- **DiskANN** [35] is the state-of-the-art disk-based graph index method, which we use as the baseline framework.
- **SPANN** [16] is a disk-based inverted index method that achieves better performance than DiskANN but requires more disk space.
- **`Starling`** is our framework, which implements all the optimizations proposed in this paper. Unless otherwise specified, we use the Vamana algorithms as the default option in `Starling`, denoted as `Starling`-Vamana or simply `Starling`.

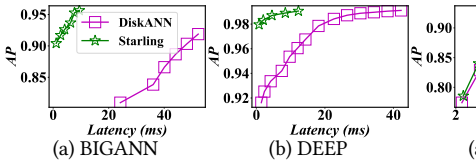
Evaluation protocol. We use *queries per second (QPS)*, *mean latency*, and *mean I/Os* to evaluate the search performance of all competitors. By default, we use eight threads on the server to serve queries and report the *QPS* based on the wall clock time from the query input to the result output. For the latency and I/Os, we record the response time and number of disk I/Os for each query and compute the *mean latency* and *mean I/Os* over all queries. For ANNS, we measure its accuracy by *Recall* (Eq. 2). Unless otherwise stated, we set $k = 10$ in *Recall*. For RS, *AP* (Eq. 3) is a more suitable accuracy metric. We fix a search radius r for each dataset following [64]. We evaluate two popular similarity metrics, *L2* and inner product (*IP*).

Segment configuration. We follow the configuration of a mainstream open-source vector database, Milvus [6, 29], with at most 2GB memory space and 10GB disk capacity for each segment by default.

Setup. We execute the C++ codes of all methods using different instances for both index building and search to align with the protocols of current vector databases [29] and related research [25, 68]. For index building, we employ a n2-standard-64 instance (ubuntu-2004-focal-v20221121, 2 TB SSD Persistent Disk) with 64 vCPUs—this facilitates fast construction and segment sharing. In terms of search functionality, vector databases typically leverage multiple smaller instances to improve query performance and promote fine-grained load balancing and scaling. In our experiments, all segments share a n2-standard-8 instance (ubuntu-2004-focal-v20221121, 375 GB NVMe Local SSD Scratch Disk) with 8 vCPUs. We use the o_direct option to read data from the disk for all competitors, circumventing operating system caching. We optimize all approaches' hyper-parameters for the segment's configuration. We report the average results of three trials on the optimum configuration.

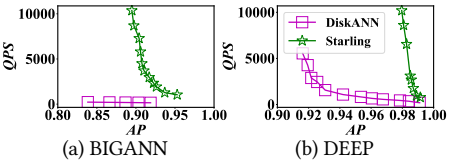
6.2 Search Performance

RS performance. Current disk-based HVSS methods largely ignore RS, except for a baseline in a competition [3]. RS support is provided by calling ANNS iteratively on DiskANN (referred to as



(a) BIGANN (b) DEEP (a) SSNPP

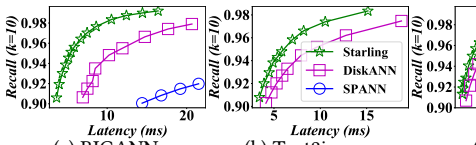
Fig. 4. RS performance (*AP* vs *Latency*).



(a) BIGANN (b) DEEP

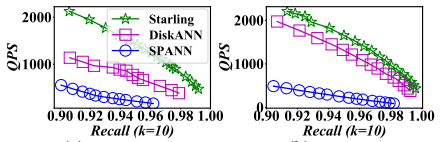
Fig. 5. RS performance (*QPS* vs *AP*).

DiskANN) [5]. Fig. 4 and 5 compare the RS performance of DiskANN and Starling. Under the same *AP*, Starling reduces query latency by up to 98% compared to DiskANN. On SSNPP, Starling exhibits a slight performance boost. We observed that most query results are near the centroid for that dataset. DiskANN's cache policy loads the data near the centroid, covering most query results. However, Starling still achieves better RS performance. Fig. 5 shows that Starling has a significantly higher throughput than DiskANN under the same *AP*. For example, it is 43.9× faster than DiskANN when $AP = 0.9$ on BIGANN. Further investigation reveals that DiskANN requires numerous disk I/Os for some queries with long RS results (e.g., 1,000). In that case, Starling achieves greater gains as it avoids more redundant disk I/Os.



(a) BIGANN (b) Text2image (a) DEEP

Fig. 6. ANNS performance (*Recall* vs *Latency*).



(a) BIGANN (b) DEEP

Fig. 7. ANNS performance (*QPS* vs *Recall*).

ANNS performance. Fig. 6 contrasts the *Recall* and *Latency* of assorted methods. SPANN on Text2image is omitted, as its latency exceeds 20ms when $Recall > 0.9$. Starling consistently delivers lower latency compared to two rivals under similar *Recall* conditions. For example, at $Recall = 0.95$ on BIGANN, Starling (5ms) achieves over 2× more speed than DiskANN (10ms) and 10× more than SPANN (50ms). Fig. 7 delineates the *QPS* and *Recall* of various methods. Starling continually exhibits superior performance over its competitors. SPANN presents lackluster results, worse than anticipated. The reason is that SPANN relies heavily on data duplication (up to eight-fold the base data size [16]). Given the segment configuration, the number of data replications is restrained, leading to a performance dip (for additional analysis, see §6.9).

6.3 I/O-efficiency

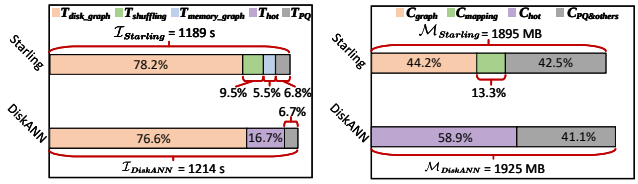
Table 2. Vertex utilization ratio (ξ) and search path length (ℓ).

Framework↓	Metric↓	BIGANN	DEEP	SSNPP	Text2image
DiskANN	ξ	0.0625	0.1429	0.1111	0.2500
Starling		**0.3438**	**0.4429**	**0.4111**	**0.8760**
DiskANN	ℓ	362	341	127	269
Starling		**182**	**240**	**100**	**167**

Vertex utilization ratio (ξ). We calculate ξ for each block loaded in the queries and average these to obtain the overall ξ for the loaded blocks. As displayed in Tab. 2, Starling consistently outperforms DiskANN across all datasets. This superior performance is credited to Starling's data locality enhancement through block shuffling, which permits a single block from the disk to encompass multiple relevant vertices. This approach heightens ξ, thereby mitigating disk bandwidth wastage. Hence, Starling exhibits higher I/O-efficiency. Nonetheless, ξ is not the exclusive factor that influences query performance–other factors such as data distribution and optimizations like the in-memory navigation graph also impact overall performance, potentially causing a non-linear relationship between ξ and query performance metrics such as latency.

Search path length (ℓ). We undertake an evaluation of ℓ for all queries, subsequently reporting the average ℓ under specified search accuracy. As denoted in Tab. 2, `Starling` exhibits a shorter ℓ compared to DiskANN. Within `Starling`, the in-memory navigation graph yields query-aware dynamic entry points that are in closer proximity to the query, thus reducing ℓ. Consequently, the IO-efficiency of `Starling` sees further improvement.

6.4 Index Cost



(a) Index processing time (b) Memory cost

Fig. 8. Index cost of two different frameworks on BIGANN.

Index processing time. The offline index process for `Starling` encompasses four components: disk-based graph construction time T_{disk_graph}, block shuffling time $T_{shuffling}$, in-memory graph construction time T_{memory_graph}, and preprocessing time for PQ short codes T_{PQ} (cf. §5.1) Consequently, the aggregate offline index procession time for `Starling` can be calculated as:

$$\mathcal{I}_{Starling} = T_{disk\_graph} + T_{shuffling} + T_{memory\_graph} + T_{PQ} \quad . \tag{8}$$

The offline index process for DiskANN comprises three parts: disk-based graph construction time T_{disk_graph}, hot vertices acquisition time T_{hot}, and preprocessing time for PQ short codes T_{PQ}. As per this structure, the total offline index processing for DiskANN is:

$$\mathcal{I}_{DiskANN} = T_{disk\_graph} + T_{hot} + T_{PQ} \quad . \tag{9}$$

Fig. 8(a) exposes the breakdown of index processing time on BIGANN. Despite the additional shuffling and construction time, `Starling` is substantially more efficient compared to DiskANN since the summation of $T_{shuffling}$ and T_{memory_graph} in `Starling` is less than T_{hot} in DiskANN. DiskANN necessitates the generation of hot vertices which involves the sampling of a large pool of queries and executing a slow disk-based graph search to tally vertex visit frequency. This procedure can be extremely time-consuming without hot vertices in memory. In contrast, `Starling`'s approach of building an in-memory navigation graph proves to be faster and more effective. It is also worth noting that both `Starling` and DiskANN have an equal T_{disk_graph} and T_{PQ}.

Memory cost. In the main memory, `Starling` manages an in-memory navigation graph (C_{graph}), the mapping of vertex IDs to block IDs ($C_{mapping}$), and PQ short codes along with other data structures ($C_{PQ\&others}$). Hence, the total memory cost of `Starling` is

$$\mathcal{M}_{Starling} = C_{graph} + C_{mapping} + C_{PQ\&others} \quad . \tag{10}$$

DiskANN, on the other hand, houses a set of hot vertices (C_{hot}) and the PQ short codes along with other data structures ($C_{PQ\&others}$). The memory cost is hence:

$$\mathcal{M}_{DiskANN} = C_{hot} + C_{PQ\&others} \quad . \tag{11}$$

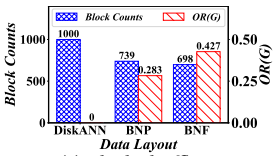
DiskANN does not maintain $C_{mapping}$ as a group of ID-contiguous vertices are allocated into a block (it locates a block by vertex ID).

Fig. 8(b) offers a comparison of the memory costs of both frameworks when handling the same set of queries with the same accuracy on BIGANN. The results reveal `Starling` has lower memory overhead compared to DiskANN. The reason for this is as follows. `Starling` requires an extra

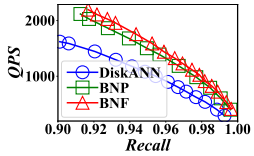
mapping of vertex IDs to block IDs post block shuffling as vertex IDs within a block are non-consecutive. In DiskANN, each hot vertex comprises vector data and the neighbor IDs of the disk-based index. Starling's in-memory graph also includes vector data and neighbor IDs but it contains 20% fewer neighbor IDs than the disk-based graph since it possesses 10% less vector data. Resultingly, $C_{graph} + C_{mapping}$ in Starling is less than C_{hot} in DiskANN.

Disk cost. Starling and DiskANN undergo the same disk cost as they utilize the same disk-based graph, albeit in different layouts.

6.5 Ablation Study

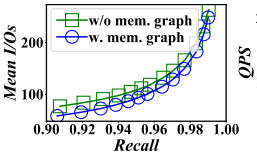


(a) Block shuffling

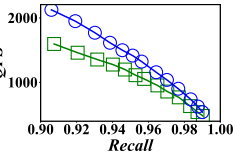


(b) Search performance

Fig. 9. Effect of block shuffling.



(a) Disk I/Os

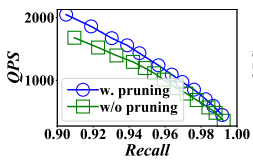


(b) Throughput

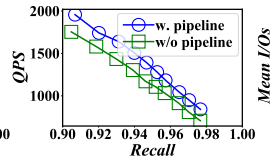
Fig. 10. Effect of in-memory graph.

Block shuffling. Fig. 9(a) reports the average number of blocks containing the top-1,000 nearest neighbors for each query within a given set (blue bar), as well as the overlap ratio $OR(G)$ (red bar) on DEEP. We limit our discussion to BNP and BNF, as BNS has slow processing times on a full-size segment. For BNF, we set the iteration parameter β to eight for an optimal balance between efficiency and efficacy. DiskANN's $OR(G)$ is found to be nearly zero, as the majority of vertices within a block are irrelevant. Both BNP and BNF consistently register higher $OR(G)$ than DiskANN, indicating the positive impact of our shuffling algorithms on data locality. BNF further amplifies $OR(G)$ on BNP, incurring a minor additional time cost, which is negligible about the total index processing time (~9.5%). Due to DiskANN's poor locality, almost all of the top-1,000 nearest neighbors are stored across disparate blocks, necessitating the checking of a minimum of 1,000 blocks for the retrieval of top-1,000 results. In comparison, BNP and BNF cut down the number of blocks by over 30%, enabling Starling to search through fewer blocks. Fig. 9(b) evidences BNP and BNF's exceptional performance in search tasks over DiskANN, with BNF outdoing BNP owing to its superior locality. By default, BNF is utilized for block shuffling in Starling.

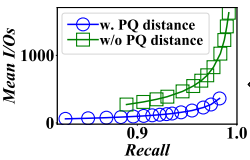
In-memory navigation graph. To ascertain the impact of the in-memory navigation graph, we turn it on/off within Starling, ensuring all other settings remain identical. As depicted in Fig. 10, this analysis highlights the changes in disk I/Os and throughput on BIGANN. The activation of the in-memory graph leads to a reduction in disk I/Os by approximately 20% for the same *Recall*. This outcome stems from initiating our search on the disk-based graph from entry points positioned closer to the query, which effectively shortens the search path length (refer to **§3.1** for further details). Fig. 10(b) reveals that the in-memory graph notably elevates throughput. Importantly, the in-memory graph implementation does not alter the vertex utilization ratio.



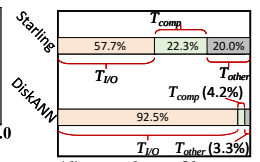
(a) Block pruning



(b) I/O-computation pipeline



(c) PQ distance



(d) Search profile

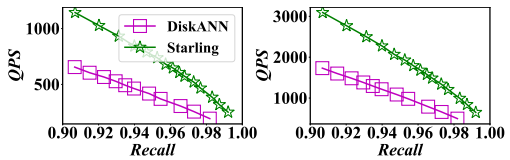
Fig. 11. Effect of the block search optimizations.

Block pruning. We assess the influence of block pruning on BIGANN by comparing `Starling` both with and without the implementation of pruning. The trade-off between *QPS* and *Recall* is presented for both scenarios. Fig. 11(a) demonstrates that `Starling`, when equipped with block pruning, exhibits superior performance. This can be attributed to the negation of unnecessary computations by pruning vertices that are distant from the query. Note that achieving an ideal graph layout with a perfect overlap ratio $OR(G) = 1$ is a challenging task (refer to **§4.1** for details). Some vertices located within a loaded block are not neighbors of the target vertex. Block pruning aids in filtering out these non-neighbor vertices, effectively circumventing unnecessary visits.

I/O and computation pipeline. Fig. 11(d) delineates the breakdown of search time for two frameworks with an identical *Recall* on BIGANN. For DiskANN, disk I/O is accountable for as much as 92.5% of the total search time, thus establishing itself as the bottleneck for HVSS on the disk-based graph. On the contrary, `Starling` enhances the vertex utilization ratio by leveraging superior data locality, thereby diminishing the I/O time ($T_{I/O}$) ratio to 57.7%. However, `Starling` also escalates the distance computation time T_{comp} since more vertices are examined in each loaded block. As a result, both I/O and computation are dominant and need to be executed in parallel for optimal performance. Fig. 11(b) outlines the impact of the I/O and computation pipeline, illustrating that the pipeline boosts *QPS* for the same *Recall*. This implementation may result in additional computations, as some vertex visitations within the current loaded block are deferred (see **§5.1** for more information). Despite this, the trade-off proves to be beneficial, enabling more efficient utilization of both the disk bandwidth and CPU, thereby improving the overall search performance.

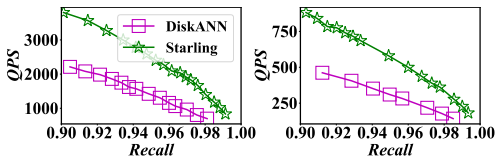
PQ-based approximate distance. Fig. 11(c) depicts the average disk I/Os before and after the implementation of the PQ-based approximate distance optimization on BIGANN for a given batch of queries. It is clear that the number of disk I/Os is significantly reduced by applying this form of optimization, while maintaining the same level of accuracy. This optimization ranks the candidate set by approximate distance, which is calculated using PQ short codes in the main memory, thereby circumventing the need for expensive disk I/O operations. Although employing the exact distance might theoretically result in enhanced upper bound accuracy, the increased disk I/O cost can be prohibitively high. With the employment of this optimization, it is possible to maintain high levels of accuracy by adjusting other parameters (such as the size of the candidate set) while limiting the amount of disk I/Os, thus making it more suitable for practical applications.

6.6 Parameter Sensitivity



(a) Threads = 4 (b) Threads = 16

Fig. 12. Effect of different threads.

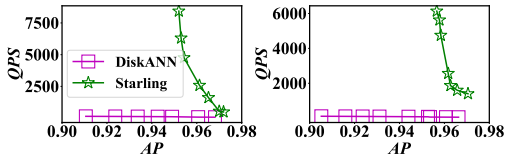


(a) k = 1 (b) k = 50

Fig. 13. Effect of different k in ANNS.

Number of threads. We carry out an evaluation of the parallelism of both `Starling` and DiskANN, employing various thread settings ranging from 4 to 16 on BIGANN. Fig.12 highlights the *QPS* vs *Recall* relationship. In both frameworks, the *Recall* remains constant with thread variations as the number of threads does not influence the search path. Notably, irrespective of the number of threads used, the *QPS* of `Starling` consistently proves to be twice as fast as DiskANN.

Number of search results. We conduct an examination on the effect of varying k from 1 to 50 on BIGANN. In Fig.13, it is clear that `Starling` exhibits a higher *QPS* than DiskANN for different values of k. This observation showcases the robustness of `Starling` in managing ANNS problems.



(a) r = 3,400 (b) r = 3,920

Fig. 14. Effect of different r in RS.

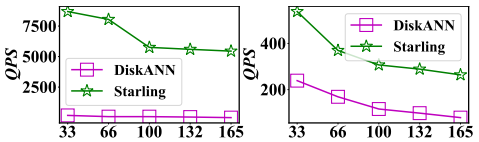
Search radius. Fig. 14 presents a comparison of the RS performance with different radii r on BIGANN. The results unequivocally demonstrate that Starling surpasses DiskANN across different r. This stark contrast highlights the superior performance of Starling in handling various r.

6.7 Scalability

Table 3. *QPS* of different number of segments on BIGANN.

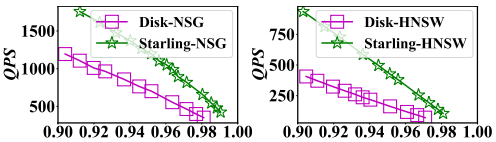
Query →	RS (*AP* = 0.90)		ANNS (*Recall* = 0.99)	
# Seg. ↓	DiskANN	Starling	DiskANN	Starling
1	181	**8,690 (48.0×)**	239	**538 (2.3×)**
2	42	**1,040 (24.8×)**	161	**321 (2.0×)**
3	31	**687 (22.2×)**	131	**257 (2.0×)**
4	23	**472 (20.5×)**	98	**193 (2.0×)**
5	19	**204 (10.7 ×)**	79	**163 (2.1×)**

Number of segments. We set a fixed segment size and manipulate the number of segments needed to process a batch of queries. Tab. 3 displays the scalability of Starling on a single machine. We report the *QPS* of both frameworks maintaining equivalent accuracy for ANNS and RS. The findings illustrate that Starling persistently surpasses DiskANN in performance for both ANNS and RS, testifying to its superior scalability in relation to the number of segments.



(a) RS (*AP* = 0.90) (b) ANNS (*Recall* = 0.99)

Fig. 15. Different data segment sizes.

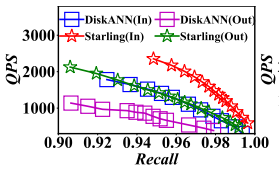


(a) NSG (b) HNSW

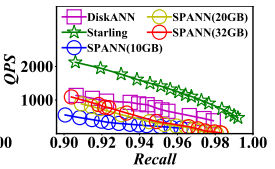
Fig. 16. Different graph algorithms.

Segment size. For other experiments, we establish a fixed dataset size of 4GB within a segment. We test Starling's performance on varying segment sizes via distinct dataset volumes. Remember, each segment's memory and disk space increase proportionately. Fig. 15 demonstrates the impact of data segment sizes. Starling sustains a higher *QPS* than DiskANN for both RS and ANNS across varying data volumes. This emphasizes that Starling scales efficiently with segment size.

Graph algorithms. By default, Starling employs the Vamana algorithm [35] to construct the disk-based graph index. Here, we assess search performance on disk with two other graph indexes, NSG [25] and HNSW [49]. We follow DiskANN to implement NSG and HNSW, but substitute NSG and HNSW (layer-0) for Vamana, resulting in Disk-NSG and Disk-HNSW. We also adapt Starling to use NSG and HNSW, obtaining Starling-NSG and Starling-HNSW. Notably, Starling-HNSW has a multi-layered in-memory navigation graph with HNSW's upper-layered graphs. Fig. 16 shows the *QPS* vs *Recall* comparison. We find that the two graph indexes on Starling perform better than the baseline framework. For example, Starling-HNSW's QPS is over 2× higher than Disk-HNSW. This shows Starling's generality to support other graph algorithms.

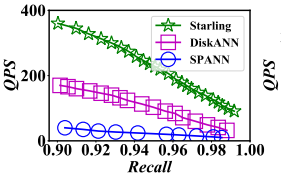


(a) Query distribution

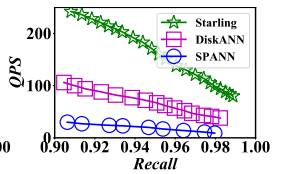


(b) Segment setup

Fig. 17. Different queries and segments.



(a) 8GB



(b) 16GB

Fig. 18. Different data sizes.

6.8 In- and Not-in-Database Queries

We compare different methods on in-database and not-in-database queries on BIGANN. In-database queries are vectors that exist in the graph index, while not-in-database queries are not. We randomly sample 10,000 vectors from the base data as in-database queries and execute the search on two frameworks. Fig. 17(a) shows that `Starling` consistently outperforms DiskANN for both types of queries. Moreover, in-database queries achieve higher throughput than not-in-database queries for both frameworks. This is because in-database queries can leverage some better query-aware entry points, which may be the query points themselves. However, the in-memory navigation graph can also find some entry points close to the not-in-database queries to shorten their search path. Notably, both types of query workloads benefit from data locality through offline block shuffling.

6.9 Evaluation on Other Segment Setups

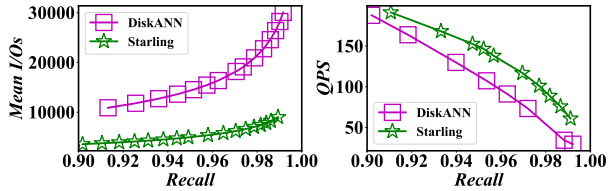
Effect of disk capacity. We fix the dataset size at 4GB and memory size at 2GB, and test different segment setups with the disk capacity from 10GB to 32GB. Fig. 17(b) shows the search performance on BIGANN. DiskANN and `Starling` exhibit the best *QPS-Recall* trade-off with an index size of less than 10GB, so we only plot one curve for each of them. SPANN improves its performance as the disk space increases, because it can duplicate more boundary data points, thereby reducing disk I/Os. However, `Starling` still performs much better than SPANN, with a smaller disk setup.
Effect of dataset size. We employ a fixed segment space with a memory of 2GB and a disk capacity of 32GB to test diverse dataset sizes of 4GB, 8GB, and 16GB. Fig. 18 shows the search performance of different methods on BIGANN (see Fig. 17(b) for the 4GB dataset). We tune the parameters of all methods to get the best *QPS-Recall* trade-off under the segment space constraint. The results conclusively depict the superior performance of `Starling` over rival methods across all dataset sizes. Importantly, the performance gap increases as the dataset size magnifies. SPANN suffers from a larger dataset size because it cannot replicate enough data to minimize disk I/Os.

6.10 Large-scale Search Results

We set the number of search results (k) to less than 50 for other experiments. In some cases, we may need much more results, such as thousands. For example, in recommendation systems, a large number of candidates are first recalled and then filtered to get the final recommendations [17, 67]. Fig. 19(a) shows the search performance with $k = 5,000$ on BIGANN. We can see that `Starling` has a much lower *Mean I/Os* than DiskANN. For instance, with a *Recall* of 0.99, `Starling` saves more than 20,000 disk I/Os per query than DiskANN. This shows that `Starling` is more efficient and effective in scenarios with large-scale search results.

6.11 Evaluation on Billion-scale Data

We evaluate our method on the one billion BIGANN dataset. We split the dataset into 31 segments, each with 2GB memory and 10GB disk. One query node has only 32GB of memory in our experiments, so we assigned 31 segments to two query nodes. We merge candidates from each segment to get the final results. We use the same setting for `Starling` and DiskANN to ensure fairness. Fig.



(a) $k = 5000$ (b) Billion-scale data

Fig. 19. Large-scale search results and Billion-scale data.

19(b) shows that Starling is more than 2× faster than DiskANN in the high recall regime (e.g., *Recall* > 0.96). Note that current vector databases use some data segmentation strategies and a query coordinator, which can avoid scanning all segments for a query [29]. Our method can work well with these strategies. For more details on data segmentation strategies, please refer to [23, 75].

7 DISCUSSION

Memory-based related work. Memory-based HVSS often involves preprocessing data to strike a balance between efficiency and accuracy. Existing algorithms fall into four categories: tree- [20, 47, 50]; quantization- [9, 30, 36]; hashing- [27, 33, 43]; and graph-based [24, 25, 49]. Tree-based and hashing-based methods are not prevalent in HVSS due to the "curse of dimensionality" and low accuracy [45]. Quantization-based methods (e.g., IVFPQ) prove efficient and memory-saving but tend to suffer from a poor recall rate[35, 60]. Graph-based methods (e.g., HNSW) exhibit leading efficiency-accuracy trade-offs. However, they necessitate both raw vector data and the graph index to be in the main memory, elevating memory consumption and impeding scalability for large-scale vectors [16]. Starling is a universal and I/O-efficient framework capable of integrating different graph algorithms (e.g., HNSW) into the disk-index component. For example, Starling accommodates HNSW seamlessly without sacrificing functionality (cf. Fig. 16(b)).

Comparison analysis with SPANN. While both Starling and SPANN leverage in-memory graphs and locality-centric disk indexes, they address varying challenges and utilize unique strategies. SPANN, a clustering-based disk index, partitions data utilizing k-means, thereby achieving an inherent locality allowing clustered data to be stored and accessed synchronously. Furthermore, it builds an in-memory graph for fast cluster retrieval. In contrast, Starling addresses disk-based graph search by optimizing data layout and search strategy. When residing in memory, graph-based methods excel in terms of accuracy and efficiency. However, once placed on disk, they incur many I/Os due to long search path and poor data locality. Starling mitigates these issues by building an in-memory graph to shorten the search path and using block shuffling to enhance locality. Note that the graph index neighborhood exhibits both similarity and navigation traits [68]. This implies that a vertex may have neighbors from different clusters [49], posing a major challenge for the locality of the graph index. We compare block shuffling with a naive strategy that assigns vertices to blocks by k-means on SSNPP. The results show that block shuffling achieves a 12× higher overlap ratio.

In-memory graph. Our in-memory graph serves as an index directing query routes, instead of acting as a cache for frequently accessed data. We initially used memory mapping (mmap) instead of direct I/O (o_direct) but found more disk I/Os linked to a memory-mapped graph. This lowers efficiency compared to the hot points strategy in the baseline framework. Our evaluation showed that the in-memory graph outperforms the hot points strategy in search performance and memory overhead. In Starling, we have the flexibility to utilize any graph algorithm like HNSW [49] or NSG [25] for the in-memory graph. We typically use the same algorithm for both in-memory and disk-based graphs. In HNSW, the upper-layer graphs are a subset of the layer-0 graph. Thus, we can keep the higher layers in memory as a multi-layered in-memory graph and store the layer-0 graph on disk. This makes HNSW easy to implement in Starling (see Fig. 16(b)).